

Strukturalni paterni

Adapter patern

Adapter patern koristi se u situacijama kada je potrebno omogućiti komunikaciju između klasa čiji interfejsi nisu međusobno kompatibilni. Njegova glavna svrha jeste prilagođavanje postojećih podataka ili funkcionalnosti bez potrebe za izmjenom originalnih klasa.

U sistemu frizerskog salona „Sakura“ adapter patern mogao bi se primijeniti prilikom prikaza rezervacija u različitim dijelovima aplikacije. Podaci o rezervaciji u modelu „Rezervacija“ organizovani su prvenstveno za internu logiku sistema, dok korisnički interfejs zahtijeva preglednije informacije poput formatiranog termina, naziva usluge i statusa rezervacije. Adapter klasa bi vršila transformaciju tih podataka u oblik prilagođen prikazu korisnicima i zaposlenicima.

Facade patern (prikazan u dijagramu)

Facade patern omogućava pojednostavljen pristup složenijem sistemu putem jedinstvene pristupne tačke. Na taj način korisnik ili drugi dijelovi sistema ne moraju direktno komunicirati sa velikim brojem različitih komponenti.

U okviru aplikacije za naš frizerski salon facade bi mogao objediniti kompletnu logiku rezervacije termina. Prilikom kreiranja rezervacije potrebno je izvršiti provjeru korisničkog računa, dostupnosti termina, evidentiranje rezervacije u bazi podataka te slanje potvrde putem e-maila.

Umjesto direktne komunikacije sa više kontrolera i modela, facade klasa bi omogućila izvršavanje svih ovih operacija kroz jedinstven i pojednostavljen interfejs.

Decorator patern

Decorator patern koristi se za proširivanje funkcionalnosti objekata u toku izvršavanja programa, bez izmjene njihove osnovne strukture.

U našem sistemu ovaj patern može biti koristan kod definisanja dodatnih opcija uz osnovne usluge salona. Na primjer, korisnik može rezervisati šišanje, a zatim toj usluzi dodati tretman za oporavak kose, stilizovanje ili korištenje premium preparata. Svaki dodatak povećava cijenu i trajanje osnovne usluge, pri čemu se funkcionalnost proširuje dinamički bez potrebe za kreiranjem posebne klase za svaku kombinaciju usluga.

Bridge patern (prikazan u dijagramu)

Bridge patern služi za razdvajanje apstraktnog dijela sistema od njegove implementacije kako bi se obje strane mogle nezavisno razvijati i proširivati.

Ovaj patern mogao bi se iskoristiti kod sistema obavještenja. Aplikacija trenutno koristi e-mail poruke za rezervacije i newsletter poruke, ali bi se u budućnosti mogli dodati i drugi načini komunikacije poput SMS ili push notifikacija. Korištenjem bridge paternna logika obavješćavanja ostala bi odvojena od konkretne implementacije načina slanja poruke. Bridge patern bi posebno bio koristan kada bi se uvele i vrste newslettera koje se ne razlikuju samo po implementaciji komunikacije, nego i po generalnoj strukturi. Korištenjem bridge paternna bi se smanjio broj naslijeđenih klasa.

Proxy patern

Proxy patern uvodi posrednički objekat koji kontroliše pristup stvarnom objektu i po potrebi vrši dodatne provjere prije izvršavanja određene operacije.

U aplikaciji frizerskog salona ovaj patern može imati važnu ulogu u zaštiti administratorskih funkcionalnosti. Prije pristupa opcijama za uređivanje proizvoda, usluga ili sadržaja newslettera, proxy objekat bi provjeravao korisničku ulogu i dozvole pristupa. Time bi se osiguralo da samo administratori mogu vršiti izmjene podataka unutar sistema.

Composite patern

Composite patern omogućava organizaciju objekata u hijerarhijsku strukturu pri čemu se pojedinačni objekti i njihove grupe tretiraju na jednak način.

U našem projektu ovaj pristup bi mogao biti iskorišten za organizaciju usluga salona. Sistem može sadržavati pojedinačne usluge, ali i pakete usluga koji se sastoje od više različitih tretmana. Na primjer, paket "Kompletna njega kose" mogao bi objediniti šišanje, feniranje i tretman kose.

Zahvaljujući composite paternu, sistem bi mogao jednako obrađivati i pojedinačne i grupisane usluge.